

Pacote FD — Auditoria de Segurança, Performance e Arquitectura

UCR · WebSockets · React Query · /admin/organizacao · 22 de Agosto de 2026

Veredicto. O isolamento de contexto (UCR) está bem desenhado na UI, mas o backend ainda autoriza pelo `users.role` do JWT. A coleção `emails` não tem índices de query. O WebSocket é um singleton sólido, mas o fan-out de subscribers e a falta de ACL nas rooms são riscos reais.

Crítica

CRÍTICA C1. Coleção `emails` sem índices de query (gargalo MongoDB)

O quê: `backend/services/db_indexes.py` cria índices para `processes`, `clients`, `UCR`, `user_email_configs` e um TTL parcial de rascunhos (`ttl_email_drafts` em `updated_at_dt` com `status: draft`). **Não existe nenhum índice operacional na coleção `emails`.**

As queries novas (Pacote DN / webmail UCR) fazem COLLSCAN + sort em memória:

Query real	Campos	Ficheiro
Lista webmail + <code>count_documents</code>	<code>\$and</code> de <code>company_id</code> / <code>account</code> (regex), <code>direction</code> , <code>status</code> , <code>is_archived</code> , <code>created_by</code> , <code>synced_for_user</code> , <code>shared_role</code> , <code>is_general</code>	<code>email_webmail.py</code>
Ordenação	<code>sent_at: -1</code>	<code>email_webmail.py</code> , <code>email_process_crud.py</code>
Timeline / processo	<code>{process_id, is_archived}</code>	<code>email_process_crud.py</code>
Dedup IMAP/Gmail	<code>message_id</code> (+ <code>shared_role</code>)	<code>email_service.py</code> , <code>gmail_api_service.py</code>
Lookup por id	<code>{id}</code>	mailbox ops, CRUD
Pesquisa	<code>\$regex</code> em <code>subject</code> , <code>body</code> , <code>from_email</code> , <code>to_emails</code>	<code>email_webmail.py</code> L426-433
Stats (7+ counts por pedido)	<code>inbox/sent/drafts/starred/trash/unread</code>	<code>email_webmail.py</code>

O endpoint de stats dispara **vários** `count_documents` na mesma request. Sem índices, cada abertura do Webmail é $O(N)$ sobre toda a mailbox.

Como corrigir:

1. Índices mínimos (compound, alinhados ao prefixo das queries):

- `{id: 1}` unique
- `{message_id: 1, account: 1}` (dedup sync)
- `{process_id: 1, sent_at: -1}` (timeline)
- `{company_id: 1, direction: 1, sent_at: -1}` (UCR mailbox)

- `{account: 1, direction: 1, sent_at: -1}` (legado sem `company_id`)
 - `{created_by: 1, sent_at: -1}` e `{synced_for_user: 1, sent_at: -1}`
 - `{shared_role: 1, sent_at: -1}` (caixa geral / indexação)
 - `{status: 1, updated_at_dt: 1}` (já coberto parcialmente pelo TTL de drafts)
2. **Não** indexar `body` com `$regex`. Pesquisa full-text → Atlas Search / índice de texto em `subject + from_email`, ou prefix search.
 3. Incluir `emails` em `get_index_stats()` — hoje a função nem lista esta coleção.
 4. Medir com `explain("executionStats")` nas queries de `run_webmail_list` e `run_webmail_stats` em staging com volume real.

CRÍTICA C2. JWT na query string do WebSocket + rooms sem ACL

O quê (token): `useWebSocket.js` `_getUrl()` constrói

`/api/ws/notifications?token=${this.token}`.

O access token aparece em access logs, proxies, browser history e referrers. `verify_websocket_token` só valida JWT; não há ticket de curta duração.

O quê (ACL): em `websocket_api_notifications.py`:

- `join_process_room` aceita **qualquer** `process_id` e adiciona o user à room `process_{id}` **sem** verificar se o utilizador tem acesso ao processo.
- `process_locked` / `process_unlocked` fazem `manager.broadcast(...)` **global** (todos os browsers ligados), com `process_id` e `user_name` controlados pelo cliente. Não há validação de posse nem rate-limit.
- Um cliente malicioso pode escutar mensagens de portal (`PORTAL_MESSAGE`) de processos alheios, ou floodar locks no Kanban de toda a empresa.

Como corrigir:

1. Autenticar o WS por `Sec-WebSocket-Protocol` ou primeiro frame pós-handshake; nunca query string. Ticket de 60s (`/api/ws/ticket`) se o proxy só aceitar query.
2. `join_process_room`: resolver o processo e aplicar a mesma regra de visibilidade que `GET /processes/{id}` (consultor/intermediário atribuído, gestão, etc.). Recusar com mensagem de erro.
3. Locks: validar `process_id`, restringir broadcast à room do processo (não global), TTL server-side, e ignorar unlocks de outro `user_id`.
4. Rate-limit de mensagens por conexão (ping excluído).

CRÍTICA C3. Backend RBAC ignora o perfil UCR activo (`effectiveRole`)

O quê: o produto 2.0 trata UCR + `X-Active-Role` / `X-Company-Id` como contexto. O gate real das rotas **não**:

```
# services/auth.py – require_roles()
user_role = user.get("role", "")
additional_roles = user.get("additional_roles", [])
if user_role == UserRole.ADMIN:
    return user # bypass total, independentemente do header
```

- `require_admin()` = JWT admin **ou** ceo (campo `users.role`), **não** o cargo da UCR activa.
- `get_effective_role()` valida `X-Active-Role` contra `users.role` + `additional_roles`, **não** contra `user_company_roles`. Um cargo que só existe na UCR é rejeitado; um `additional_roles` legado (ex.

"admin") continua a ser aceite.

- Super-admin bypass de capabilities (`role ∈ {admin, ceo}`) também é JWT-primary.

Impacto no painel `/admin/organizacao` :

- Frontend: `ProtectedRoute` usa `hasRole(user)` (JWT + `additional_roles`). `OrganizationAdminPage` usa `canAccessOrgAdmin(effectiveRole)` . **Dois gates diferentes**. Um admin JWT com perfil activo `consultor` é bloqueado na página, mas as APIs (`/admin/companies` , `/admin/user-company-roles` , `/admin/users` CRUD) continuam abertas.
- Um consultor com `additional_roles: ["admin"]` legado passa `ProtectedRoute` e as APIs `require_admin()` , mesmo com `effectiveRole === "consultor"` .

Como corrigir:

1. Fonte de verdade única: resolver (`company_id` , `role`) a partir de UCR + headers; `users.role` só como fallback de migração.
2. `require_admin()` / `require_roles()` devem consultar **o cargo efectivo** (header validado na UCR), não o documento `users` .
3. Alinhar `ProtectedRoute` com `effectiveRole` (mesmo critério que `canAccessOrgAdmin`).
4. Depreciar `additional_roles` ou sincronizá-lo automaticamente a partir da UCR no login/ `/auth/me` .
5. Testes de regressão: JWT `admin` + `X-Active-Role: consultor` → 403 em `/admin/companies` e `/admin/user-company-roles` . JWT `consultor` + UCR `admin` na empresa X → 200 só com `X-Company-Id` dessa empresa.

CRÍTICA C4. `GET /admin/users` vaza o directório completo (incluindo salário)

O quê: a rota está aberta a `admin`, `ceo`, `diretor`, `consultor`, `intermediario`, `indexacao` . Sem `for_assignment=true` , `run_get_users` devolve até **10 000** users com `UserResponse` : `email`, `telefone`, `permissions` , `base_salary` , `additional_roles` , pasta OneDrive.

A tab Utilizadores de `/admin/organizacao` chama exactamente este endpoint (`getAllAdminUsers()` → `GET /admin/users`). Qualquer consultor autenticado pode obter a mesma payload via API, mesmo sem aceder à UI.

Como corrigir:

1. Dois contratos: `GET /admin/users` (`admin/ceo`, payload completa) e `GET /users/for-assignment` (`staff`, { `id`, `name`, `role` }).
2. Remover `base_salary` e `permissions` de qualquer resposta visível a não-gestão.
3. Teste de integração: `consultor` → 403 (ou lista redacted) em `GET /admin/users` sem `for_assignment` .

CRÍTICA C5. `$regex` sem escape = ReDoS e query injection MongoDB

O quê: `sanitize_string()` remove HTML (bleach) mas **não** escapa metacaracteres de regex.

Pontos quentes:

- Webmail search: { `"subject": {"$regex": search}` } etc. (`email_webmail.py`) — qualquer staff.
- Search de emails de processo: `filters.search_term` **sem** `sanitize_string` (`email_process_crud.py` L245-251).
- Pesquisa de empresas: { `"name": {"$regex": search}` } (`companies_crud_api_list.py`) — `admin/ceo`, mas ReDoS contra a BD.

- Login: `{"email": {"$regex": "f\"^{clean_email}$", "$options": "i"}}` — um email com `.*/(a+)+` faz ReDoS no **endpoint público**.

Combinado com C1 (COLLSCAN em `body`), um `search (a+)+$` trava o processo Mongo.

Como corrigir:

1. Helper `escape_regex(s)` (`re.escape`) em **todos** os `$regex` de input de utilizador.
2. Login: query exacta case-insensitive (`email_normalized` indexado) em vez de regex.
3. Pesquisa de body: Atlas Search / prefixo; nunca regex em campo grande.
4. Timeout de query (`maxTimeMS`) nos finds de webmail.

CRÍTICA C6. `POST /set-active-company` está no router `require_admin()` e é incorrecto para multi-cargo

O quê:

- O router `user_company_roles` aplica `dependencies=[Depends(require_admin())]` a **todos** os endpoints, incluindo `POST /set-active-company`.
- `AuthContext.switchActiveCompany` chama este endpoint para **qualquer** perfil. Consultor/diretor recebem 403 (o frontend só faz `console.warn` e faz hard-reload na mesma — o backend **não** persiste `is_default`).
- `run_set_active_company` faz `update_one({"user_id", "company_id"})` sem `role`. Com Pacote EA (vários cargos na mesma empresa) marca `is_default` **no primeiro documento** que o Mongo devolver, e devolve esse `role` — pode não ser o cargo que o utilizador escolheu no ContextSwitcher.

Como corrigir:

1. Tirar `set-active-company` do router admin. Mover para `/auth/active-company` com `get_current_user`. Continuar a exigir UCR do **próprio** user.
2. Payload `{company_id, role}; update_one` com os três campos. Garantir um único `is_default=True` por `user_id`.
3. Deixar migrações (`/migrate`, `/migrate-email-configs`) só em admin e atrás de um confirm token — hoje estão no mesmo prefixo CRUD, invocáveis com um POST autenticado de admin/ceo.

Alta

ALTA A1. Índices UCR incompletos para as queries reais

O que existe (`db_indexes.py`):

- `unique (user_id, company_id, role)` — correcto para Pacote EA
- `(company_id)`
- `(user_id, is_default)` sparse

O que as queries fazem e o índice não cobre bem:

- `find_one({"id": role_id})` em CRUD/update/delete — `id` **UUID sem índice** (COLLSCAN a cada edição na tab Acessos).
- `find({"user_id"}).sort("company_name", 1)` — o `unique` composto ajuda o filtro `user_id`, mas o `sort` de `company_name` não está coberto → `SORT` em memória (limite 500).

- `find_one({"user_id", "company_id"})` (auth `get_active_company_id`, set-active, email resolver) — usa prefixo do unique **sem** `role`; com vários cargos devolve um documento arbitrário (bug funcional, não só de índice).
- `_find_ucr` fallback `_id: ObjectId` — docs legados misturam `id` UUID e `_id` Mongo.

Como corrigir:

- Unique `{id: 1}` (e backfill `id` em docs antigos).
- `{user_id: 1, company_name: 1}` se a listagem continuar a ordenar por nome.
- Deixar de resolver “empresa activa” sem `role`; o índice unique de 3 campos passa a ser o lookup canónico.
- `get_index_stats()` deve incluir `user_company_roles`.

ALTA A2. Fan-out WebSocket no frontend: renders e handlers a mais

O quê: o singleton da ligação está bem feito (refcount, heartbeat, auth close 4001/4002). O hook **não**:

1. **lastMessage no state React.** `_handleMessage` chama `_notifyStateListeners()` em **todas** as mensagens, incluindo `heartbeat` / `pong` a cada 30s. Cada instância de `useWebSocket()` faz `setLastMessage` → re-render do componente pai.
2. **Uma instância do hook = 13 handlers + 1 state listener.** Chamadas actuais no layout autenticado:
 - `DashboardLayout` (chat badge + invalidate emails)
 - `NotificationsDropdown` **duas vezes** (notificações + chat)
 - `ProcessDetails` (portal room)
 - `useKanbanRealtime` (kanban)
 - `useNewEmailRealtime` no `WebmailPage` e `onNewEmail` no layout

No Kanban+Webmail abertos: ~6–7 subscribers, ~80 closures, 6 re-renders por ping.

1. **onProcessUpdate perde o tipo de evento.** `makeHandler('onProcessUpdate')` chama `handler(payload)` sem `eventType`. `useKanbanRealtime.handleProcessUpdate(eventType, payload)` recebe o payload como `eventType` e cai no `default` — código morto. Os updates reais vêm de um **segundo** `on()` no `useEffect`, duplicando `PROCESS_CREATED/UPDATED/STATUS/ASSIGNED`.
2. **Polling fallback** (`_startPolling`): a cada 30s faz GET unread e `_dispatchEvent(NEW_NOTIFICATION)` **para cada** notificação ainda não lida. O dropdown incrementa o badge e dispara toasts em loop até o user marcar como lidas. Há **ainda** polling REST próprio no `NotificationsDropdown` (30s–5min). Três canais para o mesmo evento.
3. **AudioContext por notificação** sem `close()` — leak de contextos de áudio.
4. **_joinedRooms no singleton:** se `leaveProcessRoom` corre com WS fechado, o id fica no `set` e é re-joinado no reconnect (room fantasma + C2).

Como corrigir:

1. Um único `useWebSocket()` no `DashboardLayout` / `AuthProvider`. Filhos usam um `websocketContext` ou `queryClient + setQueryData`.
2. Não guardar `lastMessage` em state. Heartbeats não notificam React.
3. Corrigir `onProcessUpdate(type, payload)` ou remover o callback e ficar só com `on()`.
4. Desligar polling REST do dropdown quando `isConnected`; no fallback WS, dispatchar só IDs novos (`cursor created_at`).
5. Um `AudioContext` reutilizado; `close()` no unmount.

6. `leaveProcessRoom` deve remover do `Set` mesmo se `sendMessage` falhar (já remove; garantir leave no servidor via `beforeunload`).

ALTA A3. React Query: cache morto em `ProcessDetails` + `refetch` agressivo

O quê:

- `queryClient.js`: `staleTime: 60s`, `refetchOnWindowFocus: true`, `refetchOnMount: true`.
- `useProcessQuery` / `history` / `activities` / `deadlines` / `client`: `staleTime: 0`, `gcTime: 0`. Cada visita a `ProcessDetails`, cada `tab-focus` e cada `invalidateQueries` refetcha o bundle completo. O cache global é inútil nesta página — a mais pesada do CRM.
- `ProcessDetails.js` (~2700 linhas) **copia** o bundle TanStack para `useState` local (`process`, `deadlines`, `activities`, `history`) via `effects`. Qualquer `setQueryData` (ex. Kanban `PROCESS_UPDATED` que invalida `processes.detail`) dispara query update → effect → `setProcess` → segundo render. Fonte de verdade dupla, bugs de “save com estado stale” (já documentado no próprio ficheiro).
- `invalidateProcessDetailsQueries` invalida `queryKeys.processes.kanban({})` com **filtros vazios**, não o Kanban realmente montado → miss ou over-invalidation.
- `useNewEmailRealtime` faz `invalidateQueries({ queryKey: ['emails'] })` — refetch de **todas** as queries de email (webmail + processo + stats + drafts), não só a lista visível.
- Keys do org-admin (`["org-admin-companies"]`, `["org-admin-users"]`, `["org-admin-ucrs"]`) **fora** da factory `queryKeys` — inconsistente com o resto da app.

Como corrigir:

1. `ProcessDetails`: `staleTime` 30–60s, `gcTime` 5–10 min; `placeholderData` / `keepPreviousData` na navegação entre processos.
2. Eliminar o mirror `useState`; a UI lê `processBundle.process` + mutations optimistas (`useProcessMutations` já existe).
3. Invalidar Kanban com a query key real (ou `queryKey: ['processes', 'kanban']` prefix).
4. `new_email`: `setQueryData` na lista da mailbox activa, ou `invalidateQueries` com a key `queryKeys.emails.webmail(filters)`.
5. Meter keys org-admin na factory.

ALTA A4. N+1 e event-loop no caminho quente

Sítio	Problema	Correcção
<code>enrich_email</code>	2 <code>find_one</code> por email (process + user) nas listagens	<code>\$lookup</code> / <code>batch</code> <code>{"id": {"\$in": ...}}</code>
<code>run_list_companies</code>	<code>_count_company_users</code> (UCR count + fallback <code>users</code>) por empresa	um <code>aggregate</code> <code>\$group</code> por <code>company_id</code>
<code>run_webmail_stats</code>	7+ <code>count_documents</code>	uma <code>aggregation</code> <code>\$facet</code>
<code>get_active_company_id</code> (sync)	<code>loop.run_until_complete(_check())</code> dentro do event loop FastAPI	remover a variante sync; só <code>get_active_company_id_async</code>

O `run_until_complete` em contexto já `async` lança `RuntimeError` ou bloqueia o worker — cada request com `X-Company-Id` no código legado pode falhar de forma intermitente.

ALTA A5. Isolamento Webmail inconsistente (JWT vs effectiveRole)

O quê: `box=general` bloqueia por `effective_role`. `box=shared_indexacao` bloqueia por `user_role` (JWT). Um perfil UCR `indexacao` com JWT `consultor` (ou o inverso) vê/nega a caixa errada.

`can_see_all` usa `effective_role in (admin, ceo, diretor)` — correcto em espírito, mas se `get_effective_role` não lê UCR (C3), `diretor-só-UCR` nunca entra em `can_see_all`.

Como corrigir: todos os `box` gates com o mesmo resolvedor UCR; testes por combinação JWT × header × UCR.

ALTA A6. Inputs / XSS no domínio UCR e admin

- `UserCompanyRoleCreate.signature` é HTML **sem** `sanitize_html` no backend. O Webmail faz `sanitizeEmailHtml` no preview, mas o HTML cru vai para IMAP/envio.
- `EMAIL_CONFIG` no frontend permite `style` e URIs `data:` — vetor residual (CSS injection / phishing em imagem).
- `CompaniesAdminTab` valida NIF no cliente; o backend `CompanyCreate` precisa da mesma regra (não assumir só UI).
- `run_create_user`: comentário explícito “sem validação de força” na password; `generateTempPassword` no frontend não implica política no API.
- `data: dict` em `set-active-company` — sem Pydantic; aceita campos extra.

Como corrigir: sanitizar assinatura no write path (`sanitize_html` / bleach email tags); política de password no `UserCreate`; modelos Pydantic em todos os POSTs admin; NIF no `CompanyCreate`.

ALTA A7. UI /admin/organizacao vs APIs — mapa de protecção

Superfície	Gate	Avaliação
Rota React <code>/admin/organizacao</code>	<code>ProtectedRoute(admin, ceo)</code> por JWT	Furo vs perfil activo (C3)
<code>OrganizationAdminPage</code>	<code>canAccessOrgAdmin(effectiveRole)</code>	Correcto para UI
Sidebar item	<code>effectiveRole admin/ceo</code>	Correcto
<code>GET/POST /admin/companies</code>	<code>require_admin()</code> JWT	CEO ok; ignora UCR
UCR CRUD <code>/admin/user-company-roles</code>	<code>require_admin()</code> JWT	Igual; <code>set-active-company</code> não deveria estar aqui (C6)
<code>GET /admin/users</code>	staff alargado	Crítico (C4)
<code>POST/PUT/DELETE /admin/users</code>	admin/ceo JWT	OK se C3 for corrigido
<code>GET/POST/DELETE /admin/users/{id}/roles</code>	admin/ceo JWT	OK
Impersonate	admin/ceo JWT	OK; confirmar que impersonate não herda UCR do admin

Não há rota `/admin/organizacao` no FastAPI (é SPA). A segurança é 100% nas APIs acima. **Não há endpoint org-admin órfão sem auth** — o problema é o **critério** de auth, não a ausência de `Depends`.

BAIXA B1. Duplicação / código obsoleto a eliminar

Item	Problema	Ação
<code>CompaniesManagementPage.jsx</code> (~740 linhas) vs <code>CompaniesAdminTab.jsx</code>	Dois CRUDs de empresas; o <code>SystemAdminPanel</code> ainda faz embed da página antiga	Uma superfície: tab Empresas em <code>/admin/organizacao</code> . <code>System-admin</code> → redirect ou a mesma tab. Apagar a página quando o email-config IMAP da empresa viver noutra sítio (já há <code>EmailAccountsPage</code>).
<code>UsersAccessAdminTab</code> em dois sítios (<code>OrganizationAdminPage</code> e <code>SystemAdminPanel</code>)	Dois caminhos para o mesmo CRUD UCR	Só <code>/admin/organizacao?tab=utilizadores</code> . Tab “Utilizadores” do <code>system-admin</code> → <code>Navigate</code> .
<code>normalizeCompaniesPayload</code> duplicada (page + <code>organizationAdmin.js</code>)	Drift de aliases (<code>company_id</code> vs <code>id</code>)	Um helper.
<code>additional_roles</code> + UCR	Dois modelos de multi-perfil	Migração one-way: UCR canónico; campo legado read-only.
<code>ProcessDetails.js</code> monolito	Dual state + WS + AI + RGD	Extrair tabs (já no roadmap); não misturar fetch pontual com TanStack.
<code>test_db_indexes.py</code>	Monkeypatch de <code>sys.modules['services.db_indexes']</code> com um dict estático (<code>idx_email_unique</code> que não existe no serviço real)	Os testes não validam os índices de produção. Reescrever contra <code>create_indexes</code> / lista real.
Índice texto <code>processes.idx_text_search</code>	Inclui <code>personal_data.nif</code> encriptado (Fernet)	Inútil + ruído. Usar só <code>nif_hash</code> / nome.
<code>backup_restore.py</code> <code>unique</code> <code>user_email_configs</code> (<code>user_id</code> , <code>company_id</code>)	Contradiz <code>idx_user_company_email_address_unique</code> (3 campos, Pacote DN.4)	Alinhar restore ao índice actual.
Query keys org-admin ad-hoc	Fora de <code>queryKeys</code>	Factory.
Hard-reload em <code>switchActiveCompany</code>	Admite que o cache TanStack foge dados entre empresas	Preferir <code>queryClient.clear()</code> + <code>navigate</code> sem reload, depois de A3.

BAIXA B2. TTL de drafts frágil

O TTL `ttl_email_drafts` exige `updated_at_dt` datetime nativo. Inserts que só gravam `updated_at` ISO **nunca expiram**. Há migração em `diagnostics_ttl.py` mas não é automática no boot.

Como corrigir: garantir `updated_at_dt` em todo insert/update de draft; ou TTL no campo que já é escrito.

BAIXA B3. Observabilidade de índices

`get_index_stats()` omite `emails`, `user_company_roles`, `notifications`, `companies`. O seed cria `notifications (user_id, is_read)` mas `db_indexes.py` não — ambientes que só correm `create_indexes` ficam sem índice de notificações (piora A2.4).

Ordem de ataque recomendada (sem código neste pacote)

1. **Índices** `emails` + **unique** `user_company_roles.id` — ganho imediato, risco baixo.
2. **Escapar** `$regex` + **login sem regex** — fecha ReDoS público.
3. **Separar** `set-active-company` e `GET /admin/users` — fecha os furos de auth mais exploráveis.
4. `require_*` **baseado em UCR efectiva** — alinhamento real PowerCell 2.0.
5. **ACL WS + token fora da query string**.
6. **Um subscriber WS + matar** `staleTime: 0` **no ProcessDetails**.
7. **Apagar** `CompaniesManagementPage` / **tab users duplicada no system-admin**.

Nada disto exige reabrir routers gordos: os pontos de correcção estão nos serviços (`auth.py`, `db_indexes.py`, `email_webmail.py`, `websocket_api_notifications.py`, `admin_users.py`, `user_company_roles_api_*`) e nos hooks (`useWebSocket.js`, `useProcessQuery.js`, `useKanbanRealtime.js`).

Relatório gerado a partir da revisão estática do código (Pacote FD). Sem alterações de código neste pacote.